

轮趣科技

MR20 毫米波雷达网口模块 使用教程

推荐关注我们的公众号获取更新资料



版本说明:

版本	日期	内容说明
V1.0	2025/11/03	第一次发布

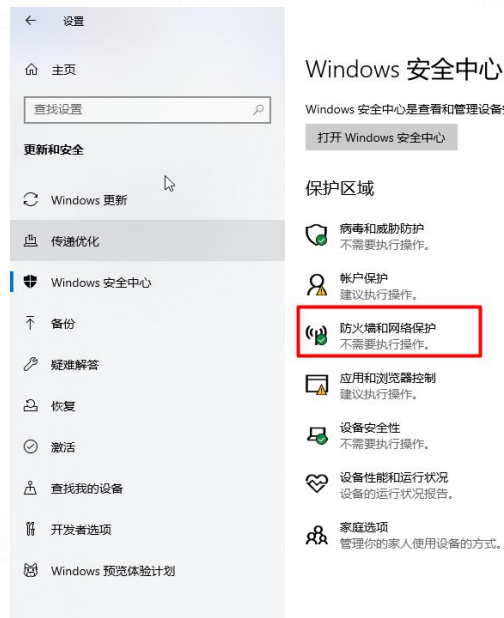
网址: www.wheeltec.net

1. 上位机软件

1.1 Windows 防火墙设置

设置防火墙，允许 MR20 上位机程序通过网络获取大量的数据包。

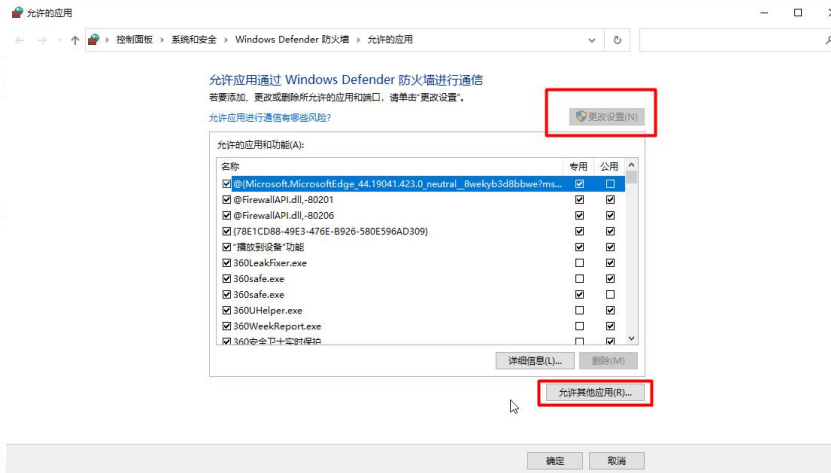
在“设置”下面的“更新和安全”中，点击“防火墙和网络保护”，进入防火墙设置界面。



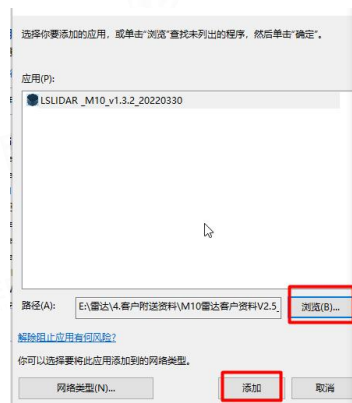
在防火墙设置界面中点击“允许应用通过防火墙”。



点击“更改设置”，然后点击“允许其他应用”。



点击“浏览”，然后选中 MR20 上位机软件中的 WHEELTEC_RADAR.exe，点击“添加”。



此时，就可以在列表中看到 MR20 上位机软件，点击确定即可。

1.2 连接电脑

把雷达连接到接线盒上，使用网线连接接线盒与电脑，然后给接线盒供电
接线盒供电范围为 5V~15V



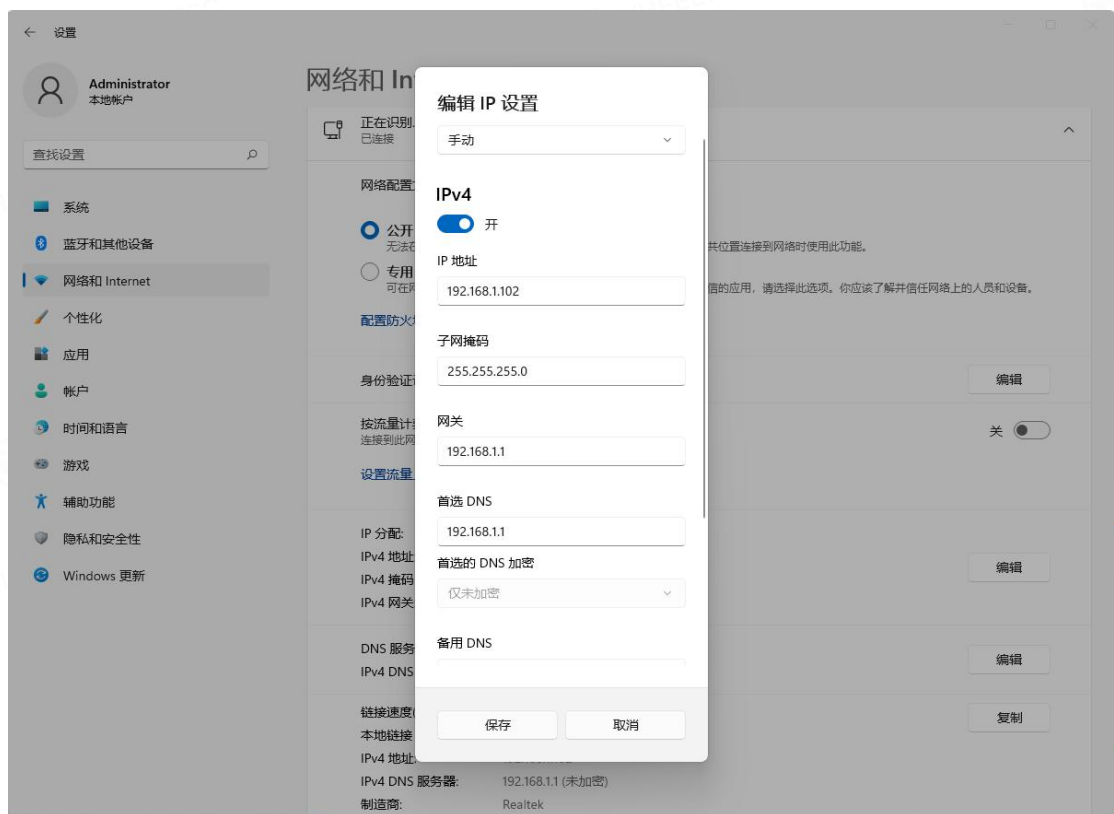
1.3 设置 IP 地址

在设置中，打开“网络和 Internet”，在雷达对应的网络下点击“属性”。



图 1.3.1 设置 IP 地址 1

然后点击“编辑”，在“编辑 IP 设置”中选择“手动”，打开“IPv4”，然后按照图片中正确修改设置，设置完成点击保存即可。



1. 4连接使用上位机

解压 windows 上位机软件中的压缩包，得到以下文件夹。

MR20毫米波雷达客户资料V1.0_20251001 > 3.Windows上位机

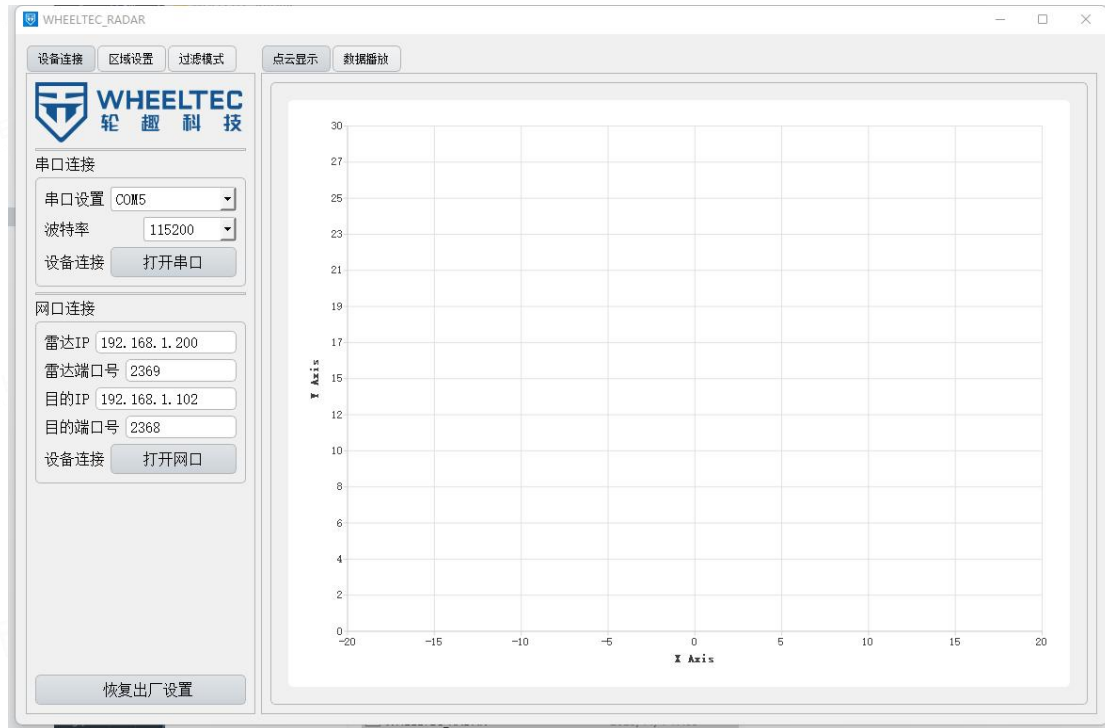
名称	修改日期	类型
 WHEELTEC_RADAR	2025/9/16 19:52	文件夹
 WHEELTEC_RADAR	2025/9/16 19:53	压缩(zip)

进入解压后的文件夹，找到 WHEELTEC_RADAR.exe 双击运行。

MR20毫米波雷达客户资料V1.0_20251001 > 3.Windows上位机 > WHEELTEC_RADAR

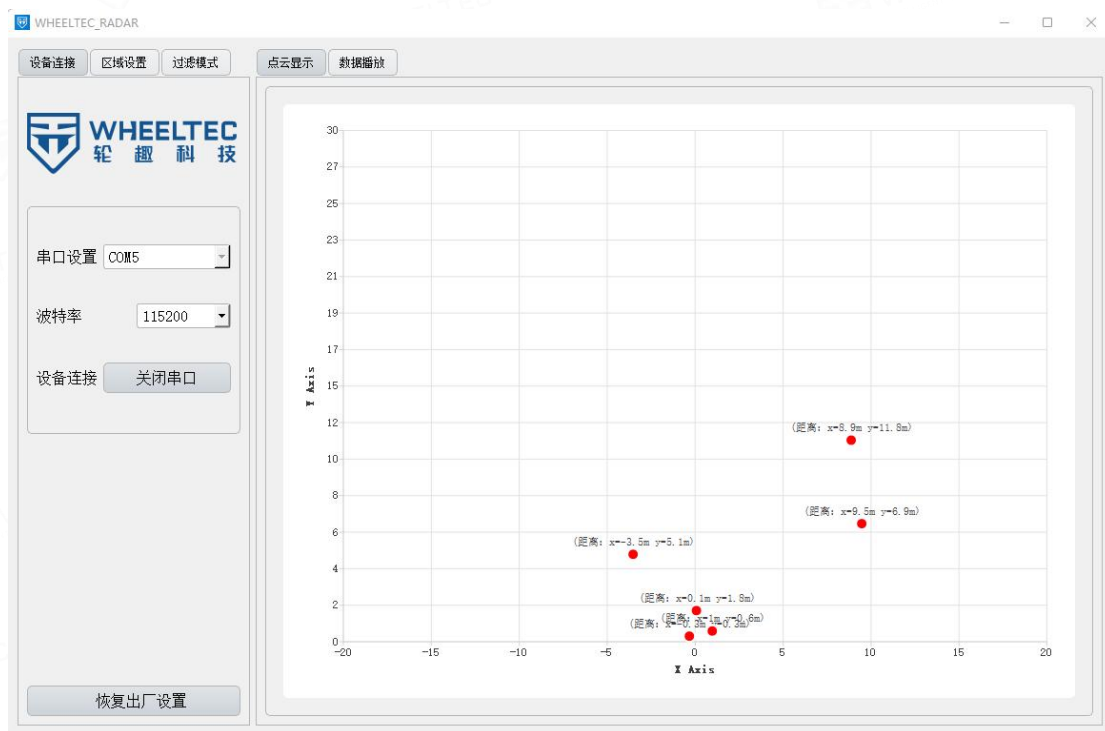
名称	修改日期	类型	大小
 iconengines	2025/9/16 19:52	文件夹	
 imageformats	2025/9/16 19:52	文件夹	
 platforms	2025/9/16 19:52	文件夹	
 translations	2025/9/16 19:52	文件夹	
 D3Dcompiler_47.dll	2014/3/11 18:54	应用程序扩展	3,386 KB
 libEGL.dll	2019/12/4 4:49	应用程序扩展	28 KB
 libgcc_s_dw2-1.dll	2015/12/29 6:25	应用程序扩展	118 KB
 libGLSV2.dll	2019/12/4 4:49	应用程序扩展	2,748 KB
 libstdc++-6.dll	2015/12/29 6:25	应用程序扩展	1,505 KB
 libwinpthread-1.dll	2015/12/29 6:25	应用程序扩展	78 KB
 opengl32sw.dll	2016/6/14 21:08	应用程序扩展	15,621 KB
 Qt5Charts.dll	2019/12/4 6:06	应用程序扩展	1,379 KB
 Qt5Core.dll	2025/9/16 19:49	应用程序扩展	6,004 KB
 Qt5Gui.dll	2019/12/4 4:49	应用程序扩展	6,086 KB
 Qt5SerialPort.dll	2019/12/4 4:57	应用程序扩展	82 KB
 Qt5Svg.dll	2019/12/4 5:00	应用程序扩展	358 KB
 Qt5Widgets.dll	2019/12/4 4:49	应用程序扩展	6,219 KB
 WHEELTEC_RADAR	2025/9/16 17:18	应用程序	133 KB

打开上位机软件后，按需求设置雷达 IP 及端口号、目的 IP 及端口号后，点击打开网口便能连上雷达。



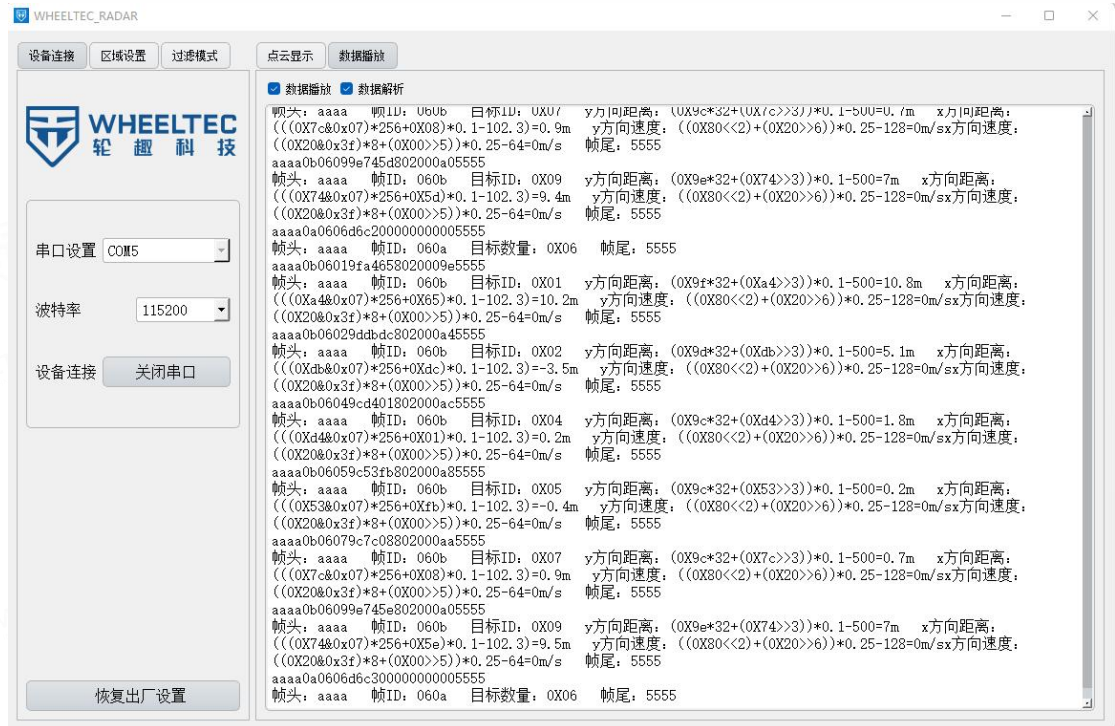
① 上位机软件功能

1) 毫米波雷达点云显示



2) 毫米波雷达数据播放及解析

通过勾选数据播放或者数据解析开启该功能

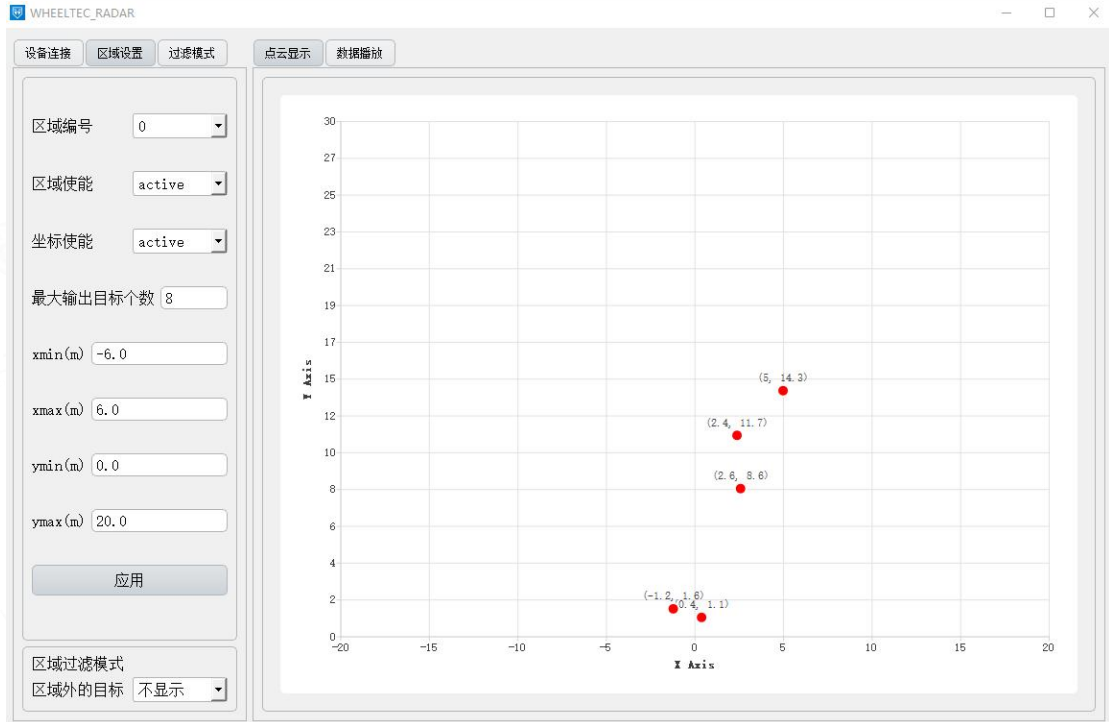


3) 区域设置

目前仅允许设置 3 个区域，设置矩形框内的允许输出目标个数为 N，如果 N 大于矩形框内的实际检测的目标个数，则输出实际检测的目标，如果 N 小于实际检测的目标个数，则输出 N 个距离最近的目标（按径向距离排序，由近及远），目前仅支持最大输出目标个数为 16 个。

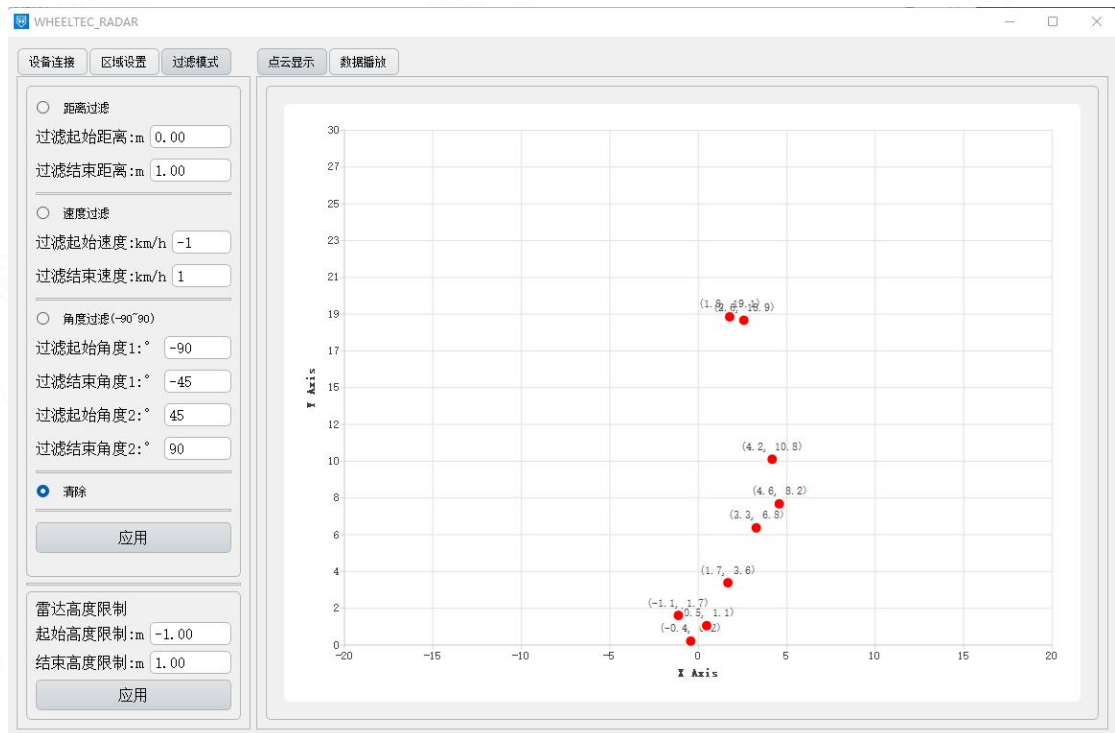
当区域使能及坐标使能同时为 active 时区域设置才会生效，否则不生效。

区域过滤功能需要配合区域设置功能，选择是否过滤掉区域设置之外的目标。



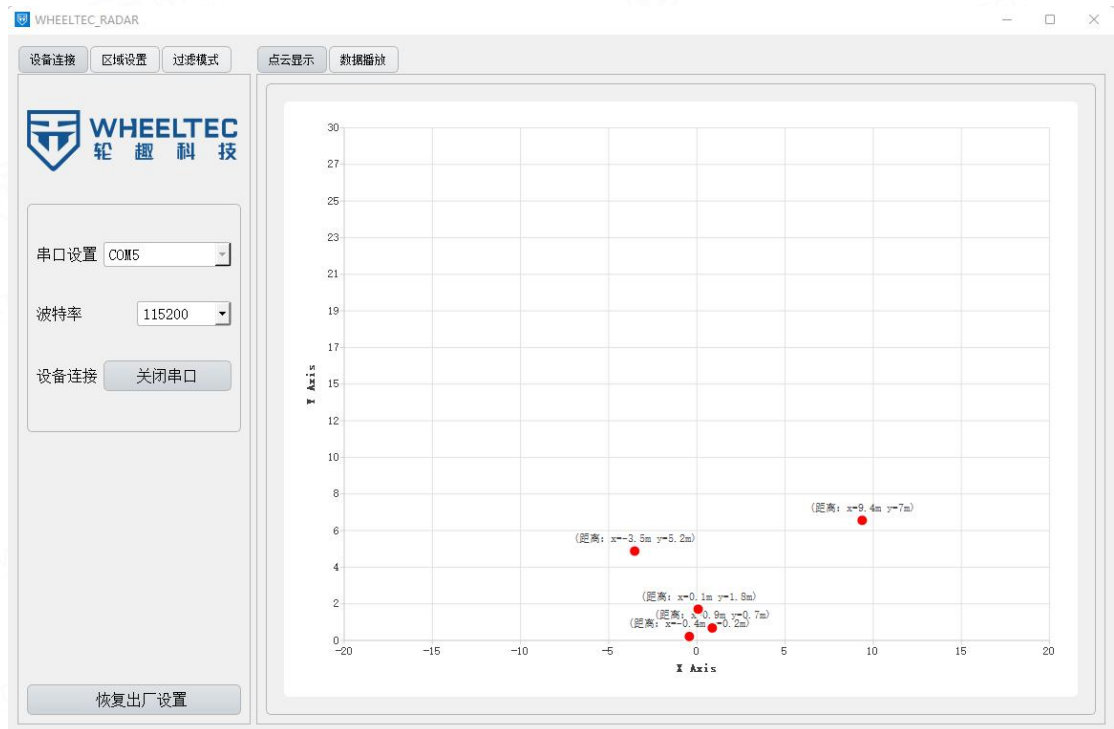
4) 过滤模式

过滤模式有四种模式，不过滤、距离过滤、速度过滤、角度过滤。选择合适的过滤模式，就能过滤掉两个参数范围内的目标（其中角度过滤以雷达前方为0度角，角度顺时针方向增加）。



雷达高度限制：限制高度范围，超出此范围的目标不显示

5) 恢复出厂设置



按下恢复出厂设置按钮即可取消所有配置，恢复出厂状态

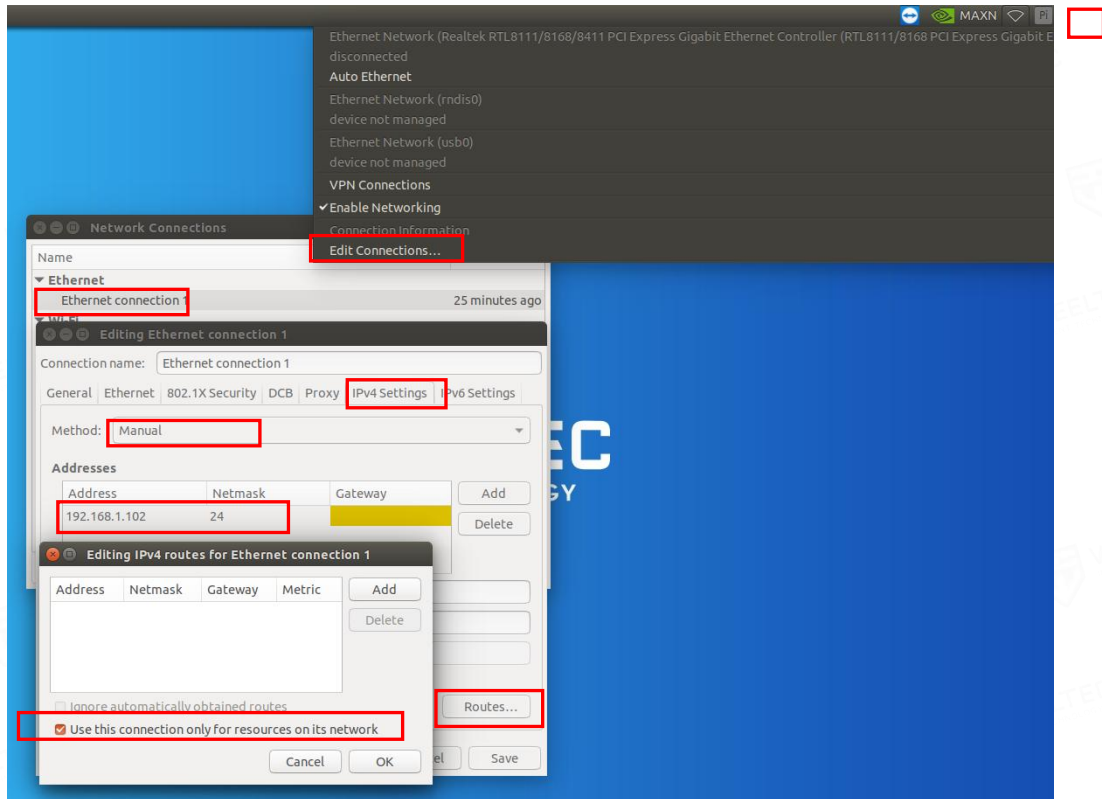
2. ROS 功能包配置

2.1 网络设置

小车通常使用 wifi 连接 PC 进行 ssh 远程登陆，小车 wifi 热点的 IP 地址默认是 192.168.0.100。

为了使小车既能 ssh 登录，又能网口传输毫米波雷达数据，我们将网口模块 IP 地址默认设置为 192.168.1.200。用户使用时，通过网线连接雷达网口模块和 ROS 主控（树莓派、nano 等）。连接后还需要在 Nano/树莓派的 Ubuntu 系统中做以下几步设置。

有线网口设置 IPv4 为同一网段。点击 Edit Connections，双击相应的有线设置，在 IPv4 Settings 中把 Method 改为 Manual，Address 设置为 192.168.1.102，Netmask 为 24，点击 Routes 并勾选 Use this connection only for resources on its network。

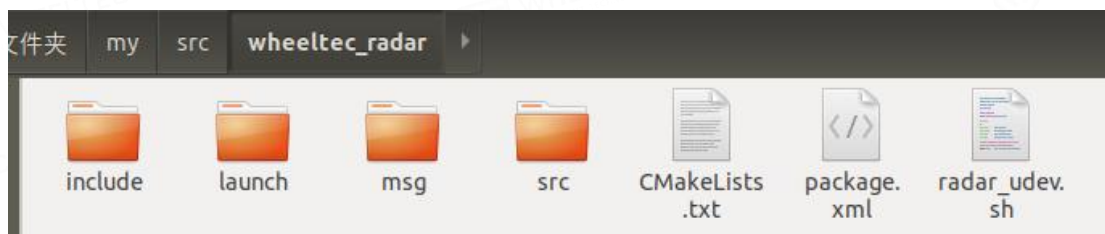


完成以上设置后可验证是否与网口模块连接成功。

```
wheeltec@wheeltec:~$ ping 192.168.1.200
PING 192.168.1.200 (192.168.1.200) 56(84) bytes of data:
64 bytes from 192.168.1.200: icmp_seq=1 ttl=64 time=1.21 ms
64 bytes from 192.168.1.200: icmp_seq=2 ttl=64 time=9.26 ms
64 bytes from 192.168.1.200: icmp_seq=3 ttl=64 time=4.99 ms
64 bytes from 192.168.1.200: icmp_seq=4 ttl=64 time=4.61 ms
64 bytes from 192.168.1.200: icmp_seq=5 ttl=64 time=1.13 ms
64 bytes from 192.168.1.200: icmp_seq=6 ttl=64 time=2.28 ms
64 bytes from 192.168.1.200: icmp_seq=7 ttl=64 time=1.05 ms
```

2. 2ROS1_SDK

进入 ROS1_SDK 目录中，解压其中的压缩包，并将我们刚刚解压得到的 wheeltec_radar 包复制到 ros 工作空间的 src 目录中。



运行以下命令编译功能包。

```
catkin_make -DCATKIN_WHITELIST_PACKAGES=wheeltec_radar
```

```
-- Found gtest sources under '/usr/src/gtest': gtests will be built
-- Found gmock sources under '/usr/src/gtest': gmock will be built
-- Found PythonInterp: /usr/bin/python2 (found version "2.7.17")
-- Looking for pthread.h
-- Looking for pthread.h - found
-- Looking for pthread_create
-- Looking for pthread_create - not found
-- Looking for pthread_create in pthreads
-- Looking for pthread_create in pthreads - not found
-- Looking for pthread_create in pthread
-- Looking for pthread_create in pthread - found
-- Found Threads: TRUE
-- Using Python nosetests: /usr/bin/nosetests-2.7
-- catkin 0.7.29
-- BUILD_SHARED_LIBS is on
-- BUILD_SHARED_LIBS is on
-- Using CATKIN_WHITELIST_PACKAGES: wheeltec_radar

-- traversing 1 packages in topological order:
- wheeltec_radar

+++ processing catkin package: 'wheeltec_radar'
==> add_subdirectory(wheeltec_radar)
-- Using these message generators: gencpp;geneus;genlisp;gennodejs;genpy
-- Configuring done
-- Generating done
-- Build files have been written to: /home/wheeltec-client/my/build

####
#### Running command: "make -j2 -l2" in "/home/wheeltec-client/my/build"
####
Scanning dependencies of target wheeltec_radar_node
[ 50%] Building CXX object wheeltec_radar/CMakeFiles/wheeltec_radar_node.dir/src/wheeltec_radar.cpp.o
[100%] Linking CXX executable /home/wheeltec-client/my/devel/lib/wheeltec_radar/wheeltec_radar_node
[100%] Built target wheeltec_radar_node
wheeltec-client@ubuntu:~/my$ catkin_make -DCATKIN_WHITELIST_PACKAGES=wheeltec_radar
```

然后我们打开路径 wheeltec_radar/launch 下的 wheeltec_radar_net.launch，一些参数配置如下图。

```
<launch>
<node name="wheeltec_radar_net_node" pkg="wheeltec_radar" type="wheeltec_radar_net_node" output="screen">
  <param name="device_ip" value="192.168.1.200"/> #雷达源IP
  <param name="device_port" value="2369"/> #雷达源端口号
  <param name="device_ip_difop" value="192.168.1.102"/> #雷达目的ip
  <param name="difop_port" value="2368"/> #雷达目的端口号
  <param name="add_multicast" value="false"/> #是否启动组播
  <param name="group_ip" value="224.1.1.2"/> #组播ip
</node>

<!-- 静态 TF 变换: base_footprint -> radar -->
<!-- <node pkg="tf2_ros" type="static_transform_publisher" name="base_to_radar" args="0 0 0 0 0 base_footprint radar" /> -->
</launch>
```

执行以下命令即可打开毫米波雷达。

```
roslaunch wheeltec_radar wheeltec_radar_net.launch
```

当启动雷达后，/wheeltec_radar_net_node 节点便会在 ros 中发布一个 /radarscan 话题和/radarpointcloud 话题以及/radartrack 话题。

```
wheeltec-client@ubuntu:~/my$ rostopic list
/radarpointcloud
/radarscan
/radartrack
/rosout
/rosout_agg
wheeltec-client@ubuntu:~/my$ rostopic info /radarscan
Type: wheeltec_radar/RadarDetectionArray

Publishers:
* /wheeltec_radar_node (http://192.168.0.136:36773/)

Subscribers: None

wheeltec-client@ubuntu:~/my$ rostopic info /radarpointcloud
Type: sensor_msgs/PointCloud2

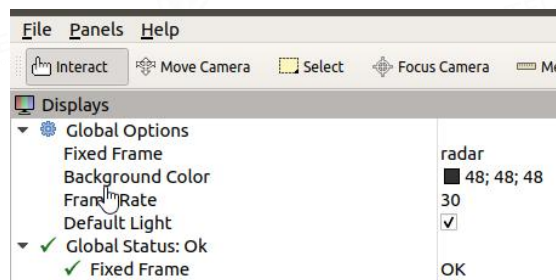
Publishers:
* /wheeltec_radar_node (http://192.168.0.136:36773/)

Subscribers: None
```

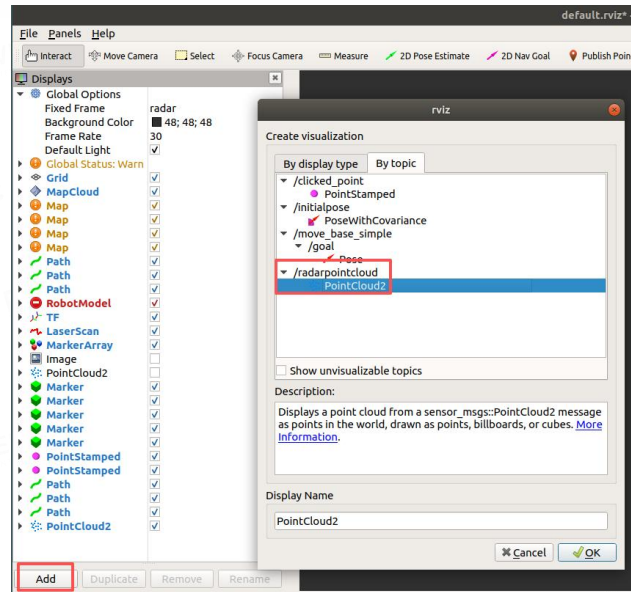
我们可以通过执行 `rostopic echo /radarscan` 命令直接进行毫米波识别目标的查看。`/radarscan` 话题中的消息类型是 `wheeltec_radar/RadarDetectionArray`，该消息类型的具体描述如下图

```
wheeltec-client@ubuntu:~/my$ rosmmsg show wheeltec_radar/RadarDetectionArray
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
wheeltec_radar/RadarDetection[] detections
  uint16 detection_id
  geometry_msgs/Point position
    float64 x
    float64 y
    float64 z
  geometry_msgs/Vector3 velocity
    float64 x
    float64 y
    float64 z
  float64 amplitude
```

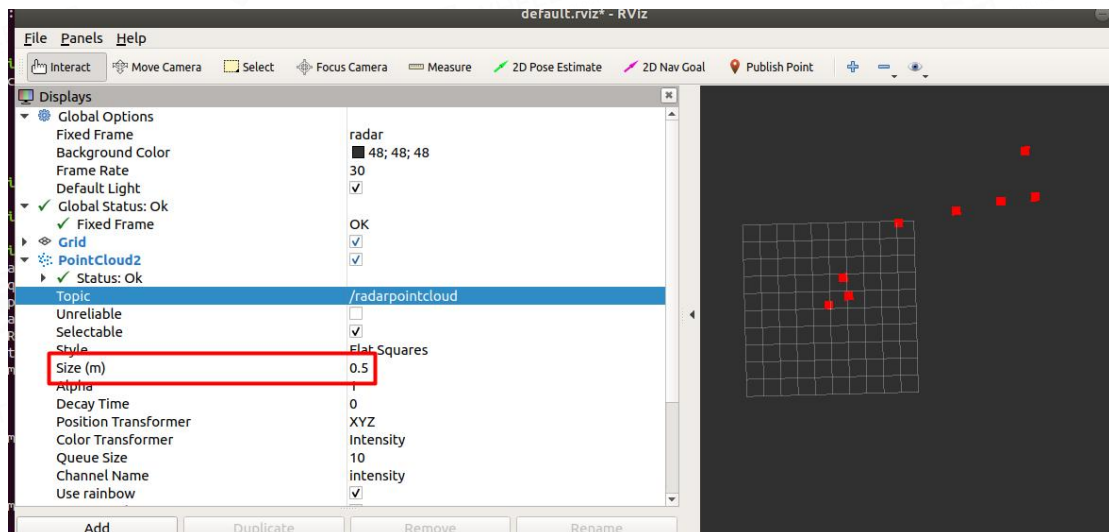
除了直接通过话题查看文本形式的雷达数据，我们也可以通过 `rviz` 来查看毫米波识别目标的图像。直接在终端输入 `rviz` 并执行。打开 `rviz` 后，先将 `Fixed Frame` 后面对应的值修改为毫米波雷达 `frame_id`: `radar`。



之后点击 `Rviz` 左下角的 `Add` 按键，在弹出的窗口中点击 `By topic` 选中 `/radarpointcloud` 话题并点击 `OK`。

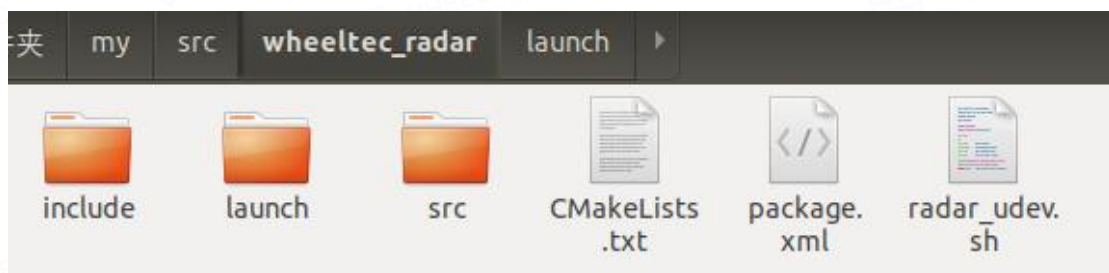


成功添加后，修改/radarpointcloud 话题的目标点云大小 size 为 0.5，我们便可以在 Rviz 中看到这样的毫米波目标点云图像。



2. 3ROS2_SDK

进入 ROS2_SDK 目录中，解压其中的压缩包，并将我们刚刚解压得到的 wheeltec_radar 包复制到 ros2 工作空间的 src 目录中。



① 运行终端并编译

安装依赖

```
sudo apt-get install ros-humble-radar-msgs
```

运行以下命令编译功能包(如果报错缺少 serial 依赖, 请将 ROS2_SDK 目录中 serial_ros2 功能包放在 src 目录中一起编译)。

colcon build

```
wheeltec@humble:~/ws$ colcon build
Starting >>> serial
--- stderr: serial
/home/wheeltec/ws/src/serial_ros2/src/impl/list_ports/list_ports_linux.cc: In function 'std::vector<std::__cxx11::basic_string<char> > glob(const std::vector<std::__cxx11::basic_string<char> >&)':
/home/wheeltec/ws/src/serial_ros2/src/impl/list_ports/list_ports_linux.cc:66:40: warning: comparison of integer expressions of different signedness: 'int' and 'size_t' {aka 'long unsigned int'} [-Wsign-compare]
   66 |         for(int path_index = 0; path_index < glob_results.gl_pathc; path_index++)
      |                                ~~~~~^~~~~~
/home/wheeltec/ws/src/serial_ros2/src/impl/list_ports/list_ports_linux.cc:57:9: warning: variable 'glob_retval' set but not used [-Wunused-but-set-variable]
   57 |         int glob_retval = glob(patterns[0].c_str(), 0, NULL, &glob_results);
      |         ~~~~~^~~~~~
/home/wheeltec/ws/src/serial_ros2/src/impl/list_ports/list_ports_linux.cc: In function 'std::string format(const char*, ...)':
/home/wheeltec/ws/src/serial_ros2/src/impl/list_ports/list_ports_linux.cc:246:31: warning: comparison of integer expressions of different signedness: 'int' and 'size_t' {aka 'long unsigned int'} [-Wsign-compare]
   246 |         else if( return_value >= buffer_size_bytes )
      |                        ~~~~~^~~~~~
---
Finished <<< serial [5.20s]
Starting >>> wheeltec_radar
--- stderr: wheeltec_radar
/home/wheeltec/ws/src/wheeltec_radar/src/wheeltec_radar.cpp: In member function 'void turn_on_radar::Get_Data()':
/home/wheeltec/ws/src/wheeltec_radar/src/wheeltec_radar.cpp:81:57: warning: suggest parentheses around '+' inside '<<' [-Wparentheses]
   81 |         radarreturn.doppler_velocity = (Receive_Data[8]<<2+(Receive_Data[9]>>6))*0.25-128;
      |                                     ~~~~~^~~~~~
/home/wheeltec/ws/src/wheeltec_radar/src/wheeltec_radar.cpp:90:50: warning: suggest parentheses around '+' inside '<<' [-Wparentheses]
   90 |         out_object.velocity.x = (Receive_Data[8]<<2+(Receive_Data[9]>>6))*0.25-128;
      |                                     ~~~~~^~~~~~
---
Finished <<< wheeltec_radar [28.9s]
Summary: 2 packages finished [34.4s]
2 packages had stderr output: serial wheeltec_radar
```

② 配置驱动包

打开功能包下的 wheeltec_radar_net.launch.py, 关于网口的配置参数如下。

```
import os
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, GroupAction, IncludeLaunchDescription
from launch.substitutions import LaunchConfiguration, PathJoinSubstitution
from launch_ros.actions import Node
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory

#def launch(launch_descriptor, argv):
def generate_launch_description():

    # Extract common parameters
    common_params = {
        'group_ip': "224.1.1.2",           #组播ip
        'add_multicast': False,           #是否启动组播
        'device_ip': "192.168.1.200",     #雷达源ip
        'device_ip_difop': "192.168.1.102", #雷达目的IP
        'device_port': 2369,              #雷达源端口号
        'difop_port': 2368,               #雷达目的端口号
    }

    turn_on_radar = Node(
        package='wheeltec_radar',
        executable='wheeltec_radar_net_node',
        name='wheeltec_radar_net_node',
        output='screen',
        parameters=[common_params],
    )
    base_to_radar = Node(
        package='tf2_ros',
        executable='static_transform_publisher',
        name='base_to_radar',
        arguments=['0', '0', '0', '0', '0', '0', 'base_footprint', 'radar'],
    )

    ld = LaunchDescription()
    ld.add_action(turn_on_radar)
    #ld.add_action(base_to_radar)

    return ld
```

③ 启动毫米波雷达

在配置完 launch 文件后，我们打开一个终端，执行以下命令：

```
ros2 launch wheeltec_radar wheeltec_radar_net.launch.py
```

成功启动后终端会出现以下字样。

```
wheeltec@humble:~/wheeltec_ros2$ ros2 launch wheeltec_radar wheeltec_radar_net.launch.py
[INFO] [launch]: All log files can be found below /home/wheeltec/.ros/log/2025-11-03-17-57-15-405882-humble-15285
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [wheeltec_radar_net_node-1]: process started with pid [15286]
[wheeltec_radar_net_node-1] [INFO] [1762163835.485485795] [wheeltec_radar_net_node]: Opening UDP socket
```

④ 雷达数据查看

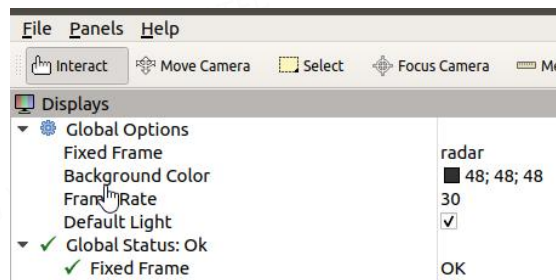
当启动雷达后，/wheeltec_radar_net_node 节点便会在 ros 中发布一个 /radarscan 话题和/radarpointcloud 话题以及/radartrack 话题。

```
wheeltec@humble:~/ws$ ros2 topic list
/parameter_events
/radarpointcloud
/radarscan
/radartrack
/rosout
/tf_static
wheeltec@humble:~/ws$ ros2 topic info /radarscan
Type: radar_msgs/msg/RadarScan
Publisher count: 1
Subscription count: 0
wheeltec@humble:~/ws$ ros2 topic info /radarpointcloud
Type: sensor_msgs/msg/PointCloud2
Publisher count: 1
Subscription count: 0
```

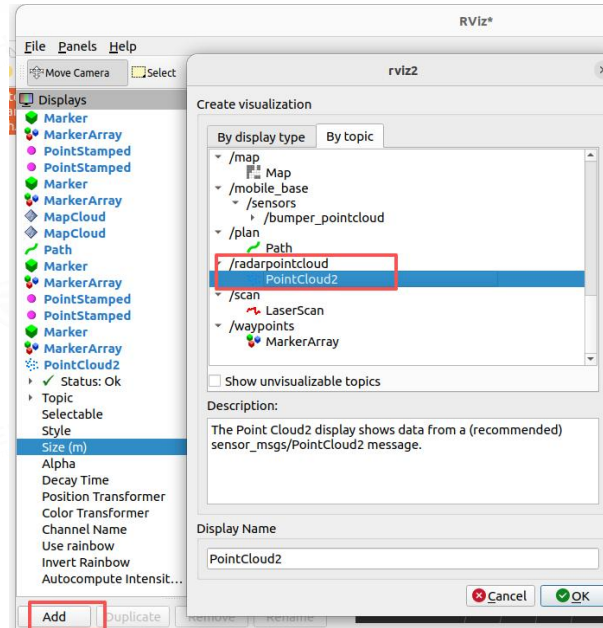
我们可以通过执行 `ros2 topic echo /radarscan` 命令直接进行毫米波识别目标的查看。`/radarscan` 话题中的消息类型是 `radar_msgs/msg/RadarScan`，该消息类型的具体描述如下图

```
wheeltec@humble:~/ws$ ros2 interface show radar_msgs/msg/RadarScan
std_msgs/Header header
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id
radar_msgs/RadarReturn[] returns
  float32 range #
  float32 azimuth # Positive angles are anticlockwise from the sensor and negative angles
clockwise from the sensor as per REP-0103.
  float32 elevation # Negative angles are below the sensor. For 2D radar, this will be 0.
  float32 doppler_velocity #
  float32 amplitude #
```

除了直接通过话题查看文本形式的雷达数据，我们也可以通过 `rviz2` 来查看毫米波识别目标的图像。直接在终端输入 `rviz2` 并执行。打开 `rviz2` 后，先将 Fixed Frame 后面对应的值修改为毫米波雷达 `frame_id: radar`。



之后点击 Rviz2 左下角的 Add 按钮，在弹出的窗口中点击 By topic 选中 `/radarpointcloud` 话题并点击 OK。



成功添加后，修改/radarpointcloud 话题的目标点云大小 size 为 0.5，我们便可以在 Rviz2 中看到这样的毫米波目标点云图像。

